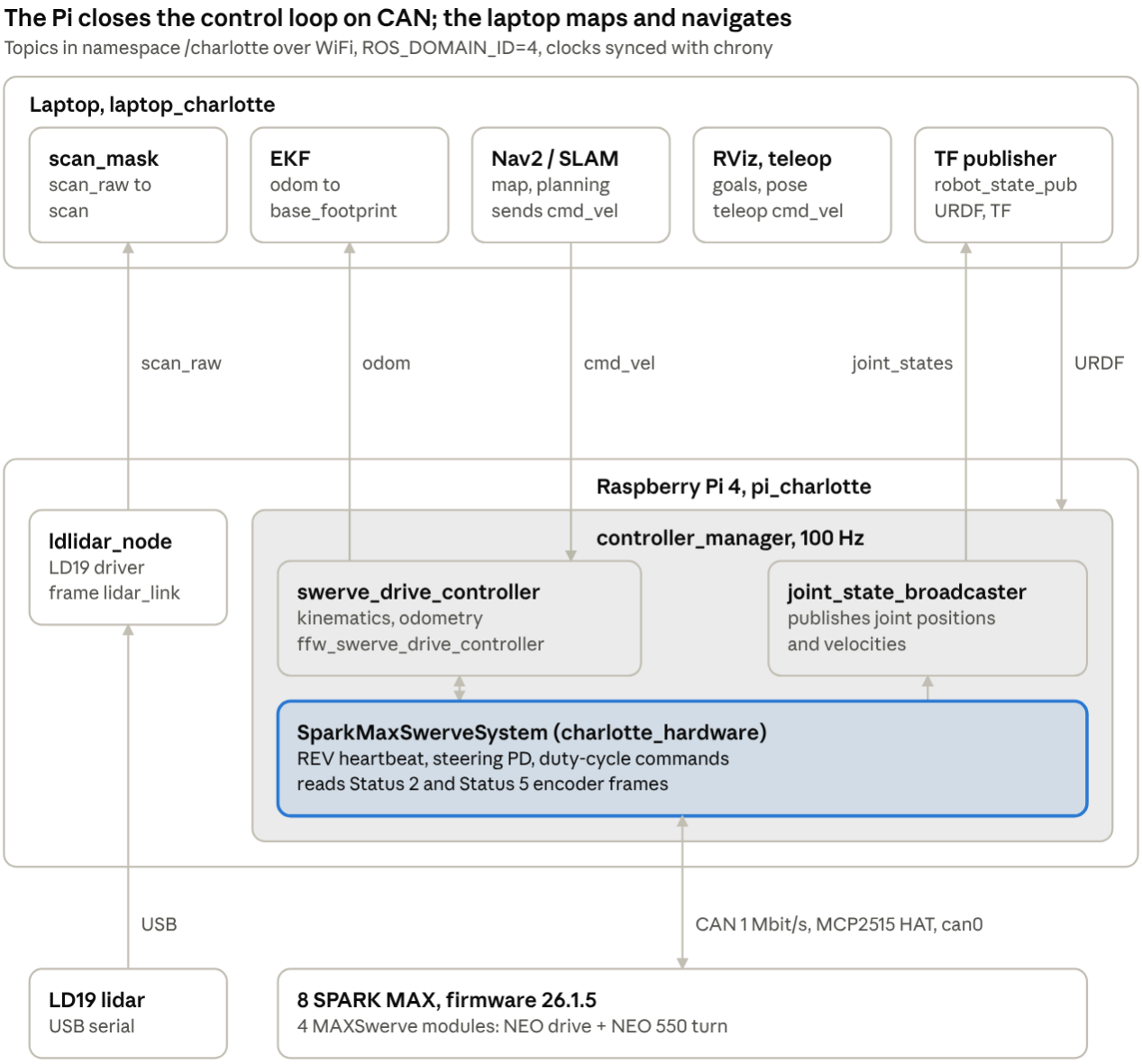
Charlotte Swerve Robot – Real Robot Bring-up Documentation

Oct 16, 2026 · @MZ

Architecture

The Pi runs everything that must react within milliseconds: the ros2_control loop at 100 Hz, the SPARK MAX CAN driver and the lidar driver. The laptop runs everything heavy: robot_state_publisher, scan mask, EKF, slam_toolbox or Nav2, RViz and teleop. Only five topics cross the WiFi, so a WiFi hiccup can delay commands but never the control loop; the controller stops the wheels after 0.5 s without `cmd_vel`.



system architecture · Pi, laptop and hardware

The URDF lives only on the laptop; the Pi's controller_manager reads it from `/charlotte/robot_description`. Controllers are spawned from the laptop, but their parameters come from the Pi's `controller_manager.yaml`.

Hardware

Four REV 3 in MAXSwerve modules (12T pinion), each with a NEO drive motor and a NEO 550 turn motor on its own SPARK MAX. All 8 SPARKs share one CAN bus with the Raspberry Pi 4.

Module	Drive (NEO) CAN ID	Turn (NEO 550) CAN ID	Through Bore Encoder on
Front left	11	12	SPARK 12
Front right	21	22	SPARK 22
Rear left	31	32	SPARK 32
Rear right	41	42	SPARK 42

Item	Detail
Robot computer	Raspberry Pi 4, Ubuntu 24.04, ROS 2 Jazzy, host `charlotte-desktop`, user `charlotte`, fixed IP
Laptop	Razer Blade 16, host `zwier-Blade-16-RZ09-0483`, user `zwier`, DHCP
CAN HAT	Waveshare RS485 CAN HAT (MCP2515, 12 MHz crystal), 1 Mbit/s
Lidar	LDRobot LD19
Gearing	Drive 5.50:1, steering 9424/203 = 46.42:1, wheel radius 0.0381 m
Network	Same WiFi, `ROS_DOMAIN_ID=4`

CAN wiring: yellow = CAN-H, green = CAN-L, daisy-chained through all SPARKs. Use the HAT's CAN H/L screw terminals, not RS485 A/B. Terminate with 120 Ω at both ends (HAT jumper "120R" plus one resistor after the last SPARK); with power off, H to L measures about 60 Ω.

Power: a 12 V bench supply browns out when several motors start at once. Use a 12 V battery with a main fuse for driving; power the Pi separately or through its own DC-DC converter.

Software packages

The colleague's `hardware_agv.cpp`, `heartbeat.py` and `motor_control.py` are replaced by `charlotte_hardware`; never run them alongside it.

Workspace	Package / file	Status	Purpose
pi_charlotte	`charlotte_hardware`	New	ros2_control SystemInterface `charlotte_hardware/SparkMaxSwerveSystem`: SocketCAN driver, heartbeat, steering PD, drive feed-forward, plus `spark_can_probe.py` (monitor, calibrate, spin)
pi_charlotte	`charlotte_pi_bringup`	New	`robot_pi.launch.py` (controller_manager + lidar), `config/controller_manager.yaml` (controller manager and swerve controller parameters), `setup_can.sh`
pi_charlotte	`ffw_swerve_drive_controller`	Copied	Same source as on the laptop; loaded by the Pi's controller_manager
pi_charlotte	`lidar_pi/bringup/lidar_pi.launch.py`	Changed	New `scan_topic` and `frame_id` arguments; robot uses `/charlotte/scan_raw` in frame `lidar_link`
pi_charlotte	`ldrobot-lidar-ros2/ldlidar_component/CMakeLists.txt`	Changed	Line 202 `install(TARGETS ${PROJECT_NAME} ldlidar` so `libldlidar.so` is installed; `component/src/ldlidar_component.cpp` line 513 counts subscribers with `_scanPub->get_subscription_count()` so the remapped topic publishes
laptop_charlotte	`swerve_robot/description/charlotte_swerve_bot.urdf`	Changed	xacro arg `sim_mode` selects the Gazebo or real `<ros2_control>` block; measured joint parameters
laptop_charlotte	`swerve_robot/bringup/robot_v2.launch.py`	Changed	Passes `sim_mode`; args `map:=` and `slam:=`; real-mode spawner without `--param-file`
laptop_charlotte	`swerve_robot/config/charlotte_slam_toolbox.yaml`	New	slam_toolbox mapping config
laptop_charlotte	`swerve_robot/package.xml`	Changed	exec_depends slam_toolbox, lifecycle_msgs, teleop_twist_keyboard

The swerve controller parameters exist twice: the sim file `swerve_robot/config/charlotte_swerve_drive_controllers_params.yaml` and the Pi's `charlotte_pi_bringup/config/controller_manager.yaml`. Change both. The Pi copy differs only in `update_rate: 100`, `use_sim_time: false` and `steering_angular_velocity_limit: 4.0`.

Raspberry Pi installation

One-time setup of the Pi (Ubuntu 24.04, ROS 2 Jazzy already installed). Paste multi-line blocks without leading spaces.

1. Update the package lists first (stale lists give 404 errors), then install:

```
sudo apt update && sudo apt upgrade
sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers ros-
jazzy-backward-ros \
  ros-jazzy-generate-parameter-library can-utils python3-can chrony rsync
```

2. Enable the CAN HAT: `sudo nano /boot/firmware/config.txt`, add under `[all]`, then `sudo reboot`:

```
dtparam=spi=on
dtoverlay=mcp2515-
can0,oscillator=12000000,interrupt=25,spimaxfrequency=2000000
```

   Check with `ip link show can0`.

3. Bring `can0` up automatically at 1 Mbit/s with a 1000-frame transmit queue:

```
printf '[Match]\nName=can0\n\n[CAN]\nBitRate=1M\nRestartSec=100ms\n' |
sudo tee /etc/systemd/network/80-can.network
printf '[Match]\nOriginalName=can0\n\n[Link]\nTransmitQueueLength=1000\n'
| sudo tee /etc/systemd/network/80-can.link
sudo systemctl enable systemd-networkd && sudo reboot
```

   After reboot `ip -details link show can0` shows `state ERROR-ACTIVE`, `bitrate 1000000`, `qlen 1000`. Manual fallback: `ros2 run charlotte_pi_bringup setup_can.sh`.

4. Time sync: `sudo nano /etc/chrony/chrony.conf`, add at the bottom, then `sudo systemctl restart chrony`:

```
server zwier-Blade-16-RZ09-0483.local iburst prefer
makestep 1 -1
```

   `chronyc sources` must show `^*` before the laptop.

5. Copy the workspace from the laptop (run on the laptop):

```
rsync -av pi_charlotte/src/ charlotte@<pi-ip>:~/pi_charlotte/src/
```

6. Apply the lidar fix if not already in the copied source (line 202 of `ldlidar_component/CMakeLists.txt` must read `install(TARGETS ${PROJECT_NAME} ldlidar)`).

7. Build (one worker to avoid running out of memory):

   ```
   cd ~/pi_charlotte && colcon build --parallel-workers 1 && source
   install/setup.bash
   ```

   The deprecation warning about `on_init` in `charlotte_hardware` is harmless.
8. Add to `~/.bashrc`: `export ROS_DOMAIN_ID=4` and `source ~/pi_charlotte/install/setup.bash`.

Laptop installation

One-time setup of the laptop (Ubuntu 24.04, ROS 2 Jazzy, Gazebo and Nav2 already used for simulation).

1. Install the extra packages:

   ```
   sudo apt update
   sudo apt install ros-jazzy-slam-toolbox ros-jazzy-teleop-twist-keyboard
   ros-jazzy-nav2-map-server chrony
   ```

2. Time server for the Pi: `sudo nano /etc/chrony/chrony.conf`, add at the bottom, then `sudo systemctl restart chrony`:

   ```
   allow <pi-ip>
   local stratum 10
   ```

   The firewall is inactive (`sudo ufw status`); if it is ever enabled, run `sudo ufw allow 123/udp`.
3. Copy `laptop_charlotte/src` from the workspace zip over `~/laptop_charlotte/src`, then build:

   ```
   cd ~/laptop_charlotte && colcon build && source install/setup.bash
   ```

4. Add to `~/.bashrc`: `export ROS_DOMAIN_ID=4` and `source ~/laptop_charlotte/install/setup.bash`.
5. Optional: REV Hardware Client 2 for SPARK MAX configuration over USB.

Simulation still works unchanged: `ros2 launch swerve_robot robot_v2.launch.py` (default `use_sim_time:=true`, so `sim_mode` selects the Gazebo hardware).

SPARK MAX configuration

All 8 SPARKs run firmware 26.1.5, configured in REV Hardware Client 2 over USB. Control runs on the Pi, so PID slots, MAXMotion, limit switches and conversion factors stay at their defaults. Full tab-by-tab detail is in the separate document *SPARK MAX Configuration – Charlotte MAXSwerve*; one correction to it: the absolute encoder is **not** inverted (the URDF uses `abs_sign -1` instead).

Setting	Drive (11, 21, 31, 41)	Turn (12, 22, 32, 42)
Motor type	NEO (Kv ≈ 480)	NEO 550
Idle mode	Brake	Brake
Inverted	Off	Off
Open / closed loop ramp rate	0	0
Smart current stall / free limit	50 A / 50 A	20 A / 20 A
Voltage compensation	Nominal, 12 V	Nominal, 12 V
Closed loop sensor	Main encoder	Main encoder
Hard limits fwd / rev	Keep moving	Keep moving
Soft limits	Off	Off
Follower mode	Off	Off
Conversion factors (all encoders)	1	1
Compatibility port config	Default	Default
Absolute encoder inverted / offset	Off / 0 (nothing connected)	Off / 0
Status 0, 1 period	100 ms	100 ms
Status 2 period	20 ms	20 ms
Status 5 period	1000 ms	100 ms
Status 3, 4, 6, 7, 8 / 9 period	1000 ms / 100 ms	1000 ms / 100 ms
Status frame force enable	0, 1, 2	0, 1, 2, 5

After any change: persist the parameters, power-cycle, and check the CAN ID. Disconnect USB before using CAN from the Pi.

Bring-up and calibration

All 8 motors are verified and calibrated. Run these on the Pi with the robot on blocks, and never while `robot_pi.launch.py` or another CAN sender is running.

1. `ros2 run charlotte_hardware spark_can_probe.py monitor --seconds 10`: 8 devices, Status 2 at ~50 Hz on all, Status 5 on the turn motors. Turning a wheel by hand changes `motor_pos_rot`.
2. Wheels straight ahead, bevel gears on the same side: `ros2 run charlotte_hardware spark_can_probe.py calibrate --steer-ids 12 22 32 42 --abs-sign -1`. Copy the offsets to the URDF.
3. Per motor: `ros2 run charlotte_hardware spark_can_probe.py spin --id <id> --duty 0.05`. Note the physical direction and the printed deltas.

Joint	CAN ID	direction	abs_sign	offset [rad]	Spin result at +5 % duty
fl_steering_joint	12	1	-1	-0.0065	CCW, motor +15.05 rot, abs −0.32
fr_steering_joint	22	1	-1	-0.0126	CCW, motor +14.98 rot, abs −0.31
rl_steering_joint	32	1	-1	-0.0067	CCW, motor +15.62 rot, abs −0.33
rr_steering_joint	42	1	-1	-0.0125	CCW, motor +14.38 rot, abs −0.31
fl_wheel_joint	11	-1			Backward, 268 RPM
fr_wheel_joint	21	-1			Backward, 268 RPM
rl_wheel_joint	31	-1			Backward, 263 RPM
rr_wheel_joint	41	-1			Backward, 263 RPM

The absolute encoder counts down while the module turns counter-clockwise (the SPARKs do not invert it), hence `abs_sign -1`. A decrease past 0 wraps to 1, so a raw `abs_pos` going 0.001 → 0.675 is a change of −0.325. Motor rotation ÷ 46.42 matched the absolute change on every module, confirming the gear ratio. The modules had been zeroed with the REV tool, so all offsets are within 1° of zero.

Other URDF hardware settings: `firmware_layout 2025`, `send_universal_heartbeat false`, `steer_control_mode software`, `drive_control_mode duty_cycle`, bench limits `steer_max_duty 0.25` and `drive_max_duty 0.3` (0.6 and 0.8 on battery).

Odometry and lidar check

Straight-line odometry is accurate to 0.6 % over 2.4 m, so `wheel_radius` stays 0.0381; the lidar was physically repositioned after its scan appeared rotated 90° clockwise and now lines up.

Method. Odometry keeps counting from the moment the launch starts and does not reset between runs. Take the start reading at the 0 mark just before driving, and the end reading after stopping, each with:

```
ros2 topic echo /charlotte/odom --once --field pose.pose
```

Distance = √(Δx² + Δy²); heading = 2·atan2(z, w) of the orientation. `A message was lost!!!` is harmless (one dropped message over WiFi). For a continuous view in metres and degrees use `ros2 run tf2_ros tf2_echo odom base_footprint --ros-args -r /tf:=/charlotte/tf -r /tf_static:=/charlotte/tf_static`. Pitfall: reusing the previous run's end value as the next start gives meaningless results, because the robot was moved back in between.

Run	Teleop speed	Odometry	Measured	Odometry / real	Heading change	Sideways drift
Forward 1 m	0.1 m/s	0.989 m	1.00 m	98.9 %	−0.6°	14 mm
Forward 1 m	0.1 m/s	0.986 m	≈ 1.02 m	≈ 96.7 %	+0.8°	15 mm
Forward 2.4 m	0.18 m/s	2.424 m	2.41 m	100.6 %	+1.3°	30 mm

The 2.4 m run is the reliable one; over 1 m, a 1–2 cm reading error is already 1–2 %. If a later precise run differs, new `wheel_radius` = 0.0381 × (real ÷ odometry), changed only in the Pi's `controller_manager.yaml`.

Still to do: the 360° rotation test (tape along the frame, one full turn with `j`, heading back within a few degrees). The sideways test is optional; it uses the same wheels and gear ratio as driving straight.

Lidar result: `/charlotte/scan_raw` arrives at 9.9 Hz on the laptop. Keep mapping speed at 0.2 m/s or less so scans do not blur.

Running the robot

Start the Pi first; its controller_manager waits for the URDF from the laptop. Emergency stop = Ctrl+C on the Pi launch: the heartbeat stops and the SPARKs disable within about 100 ms.

Pre-flight checks on the Pi

- [ ] `chronyc tracking`: System time offset well under 0.01 s
- [ ] `ip -details link show can0`: `ERROR-ACTIVE`, `bitrate 1000000`, `qlen 1000`
- [ ] No probe tool or `cansend` loop running; Hardware Client USB disconnected

Launch

1. Pi: `ros2 launch charlotte_pi_bringup robot_pi.launch.py`. Expect `Configured: all 8 SPARK MAXes report feedback`, four `steering angle at start` lines near 0°, then `Activated: heartbeat running, motors enabled`.
2. Laptop: `ros2 launch swerve_robot robot_v2.launch.py use_sim_time:=false slam:=true` (mapping) or `... use_sim_time:=false map:=$HOME/maps/lab.yaml` (navigation). Expect `Configured and activated swerve_drive_controller`.
3. Laptop, teleop:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r
cmd_vel:=/charlotte/cmd_vel -p speed:=0.1 -p turn:=0.3
```

Teleop key	Expected motion (verified)
`i` / `,`	Forward / backward, wheels straight
`J` / `L`	Modules at 90°, sideways left / right
`j` / `l`	Diamond pattern, rotate counter-clockwise / clockwise
`k`	Stop

Mapping (`slam:=true`): RViz Fixed Frame `map`, Map display `/charlotte/map`. Drive slowly through the room in loops, then save:

```
mkdir -p ~/maps
ros2 run nav2_map_server map_saver_cli -f ~/maps/lab --ros-args -r
map:=/charlotte/map
```

Navigation (`map:=$HOME/maps/lab.yaml`): set a 2D Pose Estimate in RViz, then a Nav2 Goal. AMCL starts at (0, 0, 0), so the pose estimate is always needed.

Firmware 26.1.5 CAN protocol

These were measured on Charlotte with `candump` and `cansend`; they are not all in REV's public docs. IDs are 29-bit extended: `0x02050000` (motor controller, REV) + API + device ID.

Frame	Arbitration ID	Data	Finding
REV heartbeat	`0x02052C80`	8 × `0xFF`	Enables the SPARKs (LED solid blue). Required: without it they send **no** status frames.

Frame	Arbitration ID	Data	Finding
FRC universal heartbeat	`0x01011840`		Keeps 26.1.5 SPARKs **disabled**. Never send it (`send_universal_heartbeat false`).
Duty cycle setpoint	`0x02050080` + id	float32 LE, then 4 zero bytes	Works. 0.05 = `CD CC 4C 3D` .
Status 0	`0x0205B800` + id		API class 46 (older firmware: class 6, `0x020518xx`)
Status 2	`0x0205B880` + id	velocity RPM [0:4], position rot [4:8]	Primary encoder, 2025 layout
Status 5	`0x0205B940` + id	velocity [0:4], position 0–1 rot [4:8]	Absolute encoder, turn motors only

SPARK LED: flashing blue = brake mode, no heartbeat; solid blue = enabled, zero output; flashing green/red = driving forward/reverse.

Useful manual tests:

```
# heartbeat loop (terminal 1)
while true; do cansend can0 02052C80#FFFFFFFFFFFFFFFF; sleep 0.02; done
# 5 % duty to device 11 (terminal 2), then send zero
while true; do cansend can0 0205008B#CDCC4C3D00000000; sleep 0.02; done
cansend can0 0205008B#0000000000000000
# count Status 2 frames of device 11 in 10 s (expect ~500)
timeout 10 candump can0,0205B88B:1FFFFFFF > /tmp/frames.txt; wc -l
/tmp/frames.txt
```

Problems and fixes

Every issue hit during bring-up, in the order it appeared; use it as the troubleshooting list.

Symptom	Cause	Fix
Sim launch: `name 'false' is not defined`	Nav2 bringup inherited our `slam` argument	Pass `'slam': 'False'` to the Nav2 include

Symptom	Cause	Fix
`apt install` gives many 404 errors	Stale package lists	`sudo apt update && sudo apt upgrade` first
`Cannot find device "can0"`	CAN HAT overlay missing	Add the `mcp2515-can0` overlay (Pi installation step 2)
`candump` empty, no errors	SPARKs only send while receiving a heartbeat	Hardware interface and probe tool send the REV heartbeat
Probe tool finds no frames although `candump` shows them	Firmware 26 status frames in API class 46	Code accepts class 6 and 46
Motor does not move, LED keeps flashing blue	Universal heartbeat keeps SPARKs disabled	`send_universal_heartbeat false`, REV heartbeat only
`Network is down`	`can0` not up after reboot	systemd-networkd config (Pi installation step 3)
`Transmit buffer full`	No device acknowledging (SPARKs unpowered or CAN loose)	Check 12 V and CAN wiring, then `sudo ip link set can0 down` and `setup_can.sh`
Turn motor 32 jerks ~1°, then overcurrent warning	SPARK 32 firmware/config fault (proven by swapping with 42)	Re-flash, factory reset, reconfigure
`permission denied` editing `/etc/...`	Not using sudo	`sudo nano`, or `... | sudo tee -a file`
Heredoc waits at `>` prompt	Pasted with leading spaces before `EOF`	Use the `printf ... | sudo tee` one-liners
Lidar: `libldlidar.so: cannot open shared object file`	SDK library not installed by its CMakeLists	Add `ldlidar` to `install(TARGETS ...)`, rebuild `ldlidar_component`
`Couldn't parse params file: '/tmp/tmp...'`	Laptop spawner sent a laptop-only file path to the Pi	Controller params on the Pi; real-mode spawner without `--param-file`

Symptom	Cause	Fix
`no encoder feedback ... hardware ERROR` while driving	Bench supply browned out, SPARKs reset	Battery; bench limits on duty and steering speed
Steering snaps to 90° extremely fast	Steering velocity limit 100 rad/s, duty 0.6	`steering_angular_velocity_limit 4.0`, `steer_max_duty 0.25`
`last_cmd_vel_time_ ... ref_timeout_` every 10 s	No `cmd_vel` yet	Normal; disappears with teleop or Nav2
Occasional controller_manager overrun warning	Pi without real-time kernel	Harmless when rare; lower `update_rate` to 50 if frequent

Lidar active and head spinning, but `/charlotte/scan_raw` silent everywhere: the driver only publishes while it has subscribers, and counted them with `count_subscribers("~/scan")`, which ignores the remap. Fixed by using `_scanPub->get_subscription_count()` (see Software packages); measured afterwards: 9.9 Hz on the laptop.

CAN health check: `ip -details -statistics link show can0`. Healthy = `ERROR-ACTIVE` and bus-error, error-warn, error-pass all 0. A large RX `dropped` count from earlier sessions is harmless when the frame-count test gives ~500 in 10 s (measured: 499).

Open items

The robot drives correctly on the floor, the lidar works and straight-line odometry is verified; what remains is the rotation check, the map and navigation.

- [x] ~~Lidar publishes on `/charlotte/scan_raw` (subscriber-count fix in `ldlidar_component.cpp`), 9.9 Hz~~
- [x] ~~Lidar orientation: repositioned after the scan appeared rotated 90° clockwise, with Fixed Frame `base_footprint` a wall in front now appears in front~~
- [ ] Scan mask: check `scan_mask_swerve_drive.yaml` sectors hide the real frame and wheels in `/charlotte/scan` (recheck after moving the lidar)
- [x] ~~Odometry straight line: 100.6 % over 2.4 m, `wheel_radius` unchanged~~
- [ ] Odometry rotation: one full turn with `j`, heading back within a few degrees
- [ ] Build the map with `slam:=true` and save it to `~/maps/lab`
- [ ] First Nav2 goal with `map:=$HOME/maps/lab.yaml`

- [ ] Update `dock_locations.yaml` to real map coordinates if docking is used
- [ ] Move to a 12 V battery, then restore `steer_max_duty 0.6` and `drive_max_duty 0.8`, and optionally raise `steering_angular_velocity_limit` to about 8
- [ ] Optional: add an IMU on `/charlotte/imu` (the EKF already expects it; it now runs on wheel odometry only)
- [ ] Optional: update `on_init` in `charlotte_hardware` to the new `HardwareComponentInterfaceParams` form to silence the deprecation warning